

Tour Ceylon

SOS Component Demonstration

Tourist PWA + Police Dashboard · Screen script · File map · Q&A

Project ID	R26-IT-152
Student	De Silva D.S.K. (IT22108654)
Component	SOS System (Tourist + Police)
Date	13 August 2026
Tourist app	http://localhost:5175
Police app	http://localhost:5176
API	http://127.0.0.1:5002/api

How to use this PDF while presenting

Demo has two apps: Tourist SOS (:5175) and Police Dashboard (:5176). For each screen: read “Say this”, click “Do this”, and use Files / How connected if they ask which file or how it works.

1. Big picture (30 seconds)

SOS helps a tourist get help fast. One tap sends an emergency alert with GPS location to the nearest police station. The police dashboard lets that station acknowledge, dispatch, chat, close the case, and download a PDF report. Travel login can open SOS without registering again.

Demo path: Tourist login → trigger SOS → workflow + chat → Police login → acknowledge → dispatch → close → PDF

Code path: React page → api/client.js → Flask route → routing/notify/pdf services → PostgreSQL

One sentence if they ask “what did you build?”
I built a Tourist SOS PWA and a Police Dashboard: one-tap alert with GPS, Haversine nearest-station routing, station workflow, chat, hospital dial, and PDF reports — with Travel login bridge.

2. Tech (short answers)

Tourist frontend?	React + Vite PWA — frontend/tourist — port 5175
Police frontend?	React + Vite + Leaflet — frontend/police — port 5176
Backend?	Same Flask API as Travel — backend/run.py — port 5002
Database?	PostgreSQL — tourists, incidents, stations, chat, etc.
Login?	JWT — touristToken / policeToken in localStorage
Routing method?	Haversine distance — nearest dispatchable station (not ML)
Sensitive data?	Fernet encryption for phone, passport/NIC, emergency contact
Map?	Leaflet on police IncidentDetail
PDF?	ReportLab for closed incident reports
Notifications?	Console stub by default; Twilio/SendGrid optional

3. How files connect (whole SOS system)

- **Tourist page / Police page** — what the user sees
- **api/client.js** — sends request + JWT
- **Flask route** — tourists.py / incidents.py / hospitals.py / auth.py
- **Service** — Haversine routing, notify, PDF export
- **Model / DB** — sos_models + PostgreSQL tables

frontend/tourist/	Tourist SOS PWA (alert, chat, profile)
frontend/tourist/src/pages/	Login, Home, History, Profile, Enter
frontend/tourist/src/components/	SOSButton, workflow, chat, hospital
frontend/tourist/src/hooks/	useGeolocation — one GPS read
frontend/police/	Police dashboard + map + stats
frontend/police/src/pages/	Login, Dashboard, IncidentDetail
frontend/police/src/components/	Map, chat, charts, cards
backend/app/routes/	incidents, tourists, hospitals, auth
backend/app/services/	routing.py, notify.py, pdf_export.py
backend/app/sos_models.py	SOS database tables
backend/seed_sos_data.py	Seed stations, officers, demo tourists

4. Tourist app — screen-by-screen

Start: backend python run.py · tourist npm run dev · open :5175 · seed: python seed_sos_data.py · demo tourist emma@example.com / tourist123

Screen A — Login / Register / Enter (Travel bridge)

Say this: Tourists can register here, or come from the Travel app. Enter / bridge-travel exchanges a Travel JWT for a tourist SOS session, so they do not register twice.

Do this: Login as emma@example.com / tourist123 OR open SOS from Travel and show handoff.

Frontend	Login.jsx · Register.jsx · Enter.jsx · api/client.js · sessionHandoff.js · travelBridge.js
Backend	routes/tourists.py (register, login, bridge-travel, me) · sos_models tourists
Saved	tourists table · localStorage touristToken + touristId

How connected: Login/Enter → POST /api/tourists/login or /bridge-travel → tourists.py → JWT → save touristToken → go Home.

Screen B — Home + SOS button ★ main tourist feature

Say this: This is the main tourist screen. The big SOS control lets them choose a type — general, medical, assault, theft, accident, or harassment. One tap takes GPS once (not live tracking), creates an incident, and sends it to the nearest police station.

Do this: Allow location. Tap SOS type (e.g. medical). Wait for sent overlay showing station name.

Frontend	App.jsx (triggerSOS) · Home.jsx · SOSButton.jsx · SOSSentOverlay.jsx · useGeolocation.js
Backend	routes/incidents.py · services/routing.py · notify.py · sos_models incidents + location_pings
Saved	incidents row + one location_ping · activeIncidentId in localStorage

How connected: SOSButton → App.triggerSOS → getCurrent() GPS → POST /api/incidents → incidents.py → Haversine nearest station → save incident → notify emergency contact/hotel → overlay shows station.

Screen C — Incident workflow (status tracker)

Say this: After sending SOS, the tourist sees the status change: Open → Acknowledged → Dispatched → Closed. The page polls the server every few seconds. There is no WebSocket — simple HTTP polling.

Do this: Keep Home open. After police acknowledge (next screens), show status updating here.

Frontend	IncidentWorkflow.jsx · Home.jsx polling
Backend	GET /api/incidents/<id>/status · incidents.py
Saved	incident status + timestamps (acknowledged_at, dispatched_at, closed_at)

How connected: IncidentWorkflow polls status API → incidents.py returns current status → UI steps light up.

Screen D — Incident chat (tourist side)

Say this: Tourist and the assigned police station can chat — text, and also image or voice. Messages belong to that incident only.

Do this: Send a short text message from tourist. Later show the same chat on police side.

Frontend	IncidentChat.jsx (tourist) · api/client.js
Backend	GET/POST /api/incidents/<id>/messages · chat / incidents routes · chat_messages table
Saved	chat_messages rows for that incident

How connected: IncidentChat → messages API with tourist JWT → server stores chat_messages → police app polls same endpoint.

Screen E — Call nearest hospital

Say this: If it is a medical case, the tourist can find the nearest hospital. Same Haversine idea as police routing. The app gives a dial number — hospital is not auto-dispatched.

Do this: Tap Call nearest hospital. Show hospital name + distance. Optional: show tel: link.

Frontend	CallNearestHospital.jsx · useGeolocation.js
Backend	routes/hospitals.py · GET /api/hospitals/nearest?lat=&lng= · hospitals table
Saved	Read-only lookup — no new incident field required

How connected: CallNearestHospital gets GPS → GET /hospitals/nearest → hospitals.py Haversine → returns dial_number.

Screen F — History + Profile

Say this: History shows past and active incidents. Profile holds tourist details like hotel and emergency contact. Sensitive fields are encrypted in the database with Fernet.

Do this: Open History briefly. Open Profile and point to emergency contact / hotel fields.

Frontend	History.jsx · Profile.jsx · TouristNav.jsx
Backend	GET /tourists/me/incidents · GET/PATCH profile via tourists.py · emergency_contacts
Saved	tourists + emergency_contacts (encrypted phone/email)

How connected: Profile/History pages → tourists.py with tourist JWT → read/update sos_models → encrypted PII at rest.

5. Police app — screen-by-screen

Open <http://localhost:5176> · demo officer st001 / Police001! (from seed station_credentials.csv)

Screen G — Police login

Say this: Officers log in with station credentials. The JWT is station-scoped — they only see incidents for their own station, not every station in Sri Lanka.

Do this: Login as st001 / Police001!. Land on Dashboard.

Frontend	police Login.jsx · api/client.js
Backend	POST /api/auth/login · officers table
Saved	localStorage policeToken · officer station_id in JWT claims

How connected: Login → /api/auth/login → officer JWT with station_id → Dashboard loads only that station's queue.

Screen H — Dashboard queue + stats ★ main police feature

Say this: This is the station queue. New SOS cases for this station appear here. Officers can see type, status, and response timing. Charts show volume and response times. Audio alert can play for new open cases.

Do this: Show the open incident from the tourist demo. Point at IncidentCard and StatsCharts.

Frontend	Dashboard.jsx · IncidentCard.jsx · StatsCharts.jsx · ResponseClock.jsx
Backend	GET /api/incidents · GET /api/incidents/stats · incidents.py
Saved	Reads incidents filtered by officer station_id

How connected: Dashboard polls GET /incidents with police JWT → incidents.py filters by station → cards + stats charts.

Screen I — Incident detail + map + workflow buttons

Say this: Opening a case shows tourist details (decrypted for officers), GPS on a Leaflet map, distance to station, and buttons: Acknowledge → Dispatch → Close. Status updates are what the tourist workflow tracker is waiting for.

Do this: Open the incident. Click Acknowledge, then Dispatch. Show map pin. Later Close.

Frontend	IncidentDetail.jsx · LiveMap.jsx · IncidentChat.jsx (police)
Backend	PATCH ../acknowledge · ../dispatch · ../close · incidents.py
Saved	incident status + acknowledged_at / dispatched_at / closed_at

How connected: Buttons → PATCH status routes → update incident row → tourist polling sees new status · map uses initial_lat/lng.

Screen J — Police chat + PDF report

Say this: Police can reply in the same incident chat. After Close, they can download a PDF report with tourist info, GPS, timestamps, and notes — useful for records.

Do this: Reply in chat. Close incident. Download report PDF.

Frontend	IncidentChat.jsx · IncidentDetail PDF button
Backend	messages APIs · GET /api/incidents/<id>/report.pdf · services/pdf_export.py
Saved	chat_messages · PDF generated from closed incident only

How connected: Close → status closed → report.pdf → pdf_export.build_incident_pdf(incident) → download.

6. Full file map

6.1 Tourist frontend (frontend/tourist/src/)

main.jsx / App.jsx	App shell; SOS trigger; auth gate; overlay
api/client.js	Tourist API + session helpers
api/sessionHandoff.js / travelBridge.js	Travel → SOS login bridge
hooks/useGeolocation.js	One-shot browser GPS
pages/Login.jsx / Register.jsx / Enter.jsx	Auth + handoff entry
pages/Home.jsx	Main screen + active incident
pages/History.jsx	Past incidents
pages/Profile.jsx	Tourist + emergency profile
components/SOSButton.jsx	Six-type one-tap SOS bar
components/SOSSentOverlay.jsx	Sent confirmation + station
components/IncidentWorkflow.jsx	Open→Ack→Dispatch→Closed tracker
components/IncidentChat.jsx	Tourist chat UI
components/CallNearestHospital.jsx	Nearest hospital dial
components/TouristNav.jsx	Bottom / nav links

6.2 Police frontend (frontend/police/src/)

App.jsx	Routes; policeToken gate
api/client.js	Police API + JWT
pages/Login.jsx	Officer login
pages/Dashboard.jsx	Station incident queue
pages/IncidentDetail.jsx	Case detail, actions, PDF
components/IncidentCard.jsx	Queue card
components/LiveMap.jsx	Leaflet map for GPS
components/IncidentChat.jsx	Police chat UI
components/StatsCharts.jsx	Volume / response charts
components/ResponseClock.jsx	Timing display

6.3 Backend SOS files

routes/tourists.py	Register, login, bridge-travel, me, profile
routes/incidents.py	Create SOS, status, workflow PATCH, stats, PDF
routes/hospitals.py	Nearest hospital lookup
routes/auth.py	Police officer login
routes/chat.py / messages in incidents	Incident chat send/list
services/routing.py	Haversine nearest station / hospital
services/notify.py	Emergency contact + hotel notify (stub/real)
services/pdf_export.py	Closed incident PDF
sos_models.py	tourists, contacts, stations, officers, hospitals, incidents, pings, chat
seed_sos_data.py	Seed Excel stations/hospitals + demo users

7. How nearest-station routing works (simple)

1. GPS	Browser gives lat/lng once at SOS tap	useGeolocation.js
2. Stations	Load dispatchable police desks with coordinates	stations table
3. Distance	Haversine great-circle distance to each station	routing.py
4. Pick one	Choose nearest station only (not broadcast all)	incidents.py
5. Save	Store station_id + distance_km on incident	incidents table
6. Notify	Alert emergency contact + hotel (console/SMS/email)	notify.py

Important for panel
No machine learning and no LLM. Routing is math (Haversine). No live tracking after trigger — privacy by design. Updates use HTTP polling, not WebSockets.

8. Seed data

Police stations	Excel with coordinates	Real reference desks
Hospitals	Excel with coordinates / hotlines	Gov + private sheets
Officer logins	station_credentials.csv	e.g. st001 / Police001!
Demo tourists	seed script	e.g. emma@example.com / tourist123

9. Questions they may ask — short answers

Is routing AI/ML?	No. Haversine distance to nearest dispatchable station.
Why only one station?	By design — nearest desk only, not spam every station.
Live GPS tracking?	No. One location ping at trigger. Extra ping API returns 409.
How status updates?	HTTP polling every few seconds — no WebSockets.
Who sees the case?	Only officers of the assigned station (station-scoped JWT).
Are phones encrypted?	Yes — Fernet at rest for phone, passport/NIC, emergency contact.
Hospital auto-called?	No — tourist gets nearest number and dials manually.
SMS/email real?	Default NOTIFY_MODE=console stub; Twilio/SendGrid optional.
Travel connection?	POST /tourists/bridge-travel exchanges Travel JWT for SOS session.
Ports?	Tourist 5175 · Police 5176 · API 5002
PDF when?	Only after incident is Closed — pdf_export.py
Chat media?	Text, image, and voice supported on incident chat
Incident types?	general, medical, assault, theft, accident, harassment
Demo accounts?	Tourist emma@example.com / tourist123 · Police st001 / Police001!
Same backend as Travel?	Yes — unified Flask app; SOS tables + travel tables together.

10. How to run before demo

- Terminal 1: `cd backend → .venv → python run.py`
- Seed once: `python seed_sos_data.py`
- Terminal 2: `cd frontend/tourist → npm run dev → http://localhost:5175`
- Terminal 3: `cd frontend/police → npm run dev → http://localhost:5176`
- Or use root frontend npm scripts if configured for both apps
- Health: `http://127.0.0.1:5002/api/health`

11. Suggested live order (8–10 min)

- Tourist login → explain JWT
- Trigger medical SOS → show overlay + station name + GPS used once
- Show workflow Open
- Police login → dashboard card appears
- Acknowledge → Dispatch → show tourist status updating
- Chat both sides · show map on police detail
- Call nearest hospital on tourist
- Close → download PDF report
- Mention Travel bridge + encryption + no ML

12. Closing line

Say at the end

So overall: the tourist app captures one GPS point and creates an incident; routing.py picks the nearest station; the police app runs Acknowledge → Dispatch → Close with chat and map; PDF export records the closed case — all through the shared Flask API and PostgreSQL.

Project R26-IT-152 · De Silva D.S.K. (IT22108654) · SOS System · 13 August 2026